

DEVOPS MASTER COURSE

Volume 8C

Google Cloud Data Platforms

Chapters 21-30

Relational Databases • Globally Distributed Data • NoSQL • Caching • Analytics • Lakehouse •
Streaming • Spark • Migration • AI-Ready Governance

2026 Edition

About This Part

Volume 8C develops the database and data-platform judgment required for production Google Cloud engineering. The objective is not to memorize product names. It is to select data services from workload evidence, design schemas and partitions that scale, protect data paths, operate recovery, and create analytics systems that remain governable when they become inputs to machine learning and generative AI.

Every chapter connects architecture to operations. Labs require validation evidence, failure testing, cost cleanup, and explicit rollback. Managed services remove infrastructure work, but they do not remove responsibility for data modeling, access control, capacity, observability, retention, lineage, or business continuity.

Volume 8 Progress

Part	Coverage	Status
8A	Foundations: hierarchy, IAM, tools, compute, VPC, billing	Complete
8B	Core infrastructure: fleets, networking, delivery, storage, connectivity, DR	Complete
8C	Databases, analytics, data engineering, AI-ready platforms, migration	This book
8D	GKE, Cloud Run, functions, APIs, Pub/Sub, Eventarc, workflows	Next
8E	Terraform, CI/CD, observability, security, governance, FinOps	Planned
8F	Enterprise projects, incidents, certifications, interviews, final capstone	Planned

Data Safety and Cost Controls

- Use an isolated learning project, budget alerts, short retention, small capacity, and explicit deletion steps. Database, BigQuery, Dataflow, Spark, replication, and network resources can continue charging after a lab ends.
- Never load real personal, payment, health, credential, or regulated data into training resources. Generate synthetic records and document the data classification.
- Prefer private connectivity, managed identities, IAM groups, least privilege, Secret Manager, customer-managed encryption only where governance requires it, and audit logging appropriate to the risk.
- Distinguish high availability, read scaling, backup, point-in-time recovery, replication, and disaster recovery. Each solves a different failure mode and must be tested independently.
- Treat deletion protection, retention, lifecycle rules, schema changes, and migration cutovers as controlled changes with owners, evidence, and rollback.

Table of Contents

Chapter	Title
21	Cloud SQL and AlloyDB Production Engineering

Chapter	Title
22	Spanner Global Relational Engineering
23	Firestore Document Database Engineering
24	Bigtable and Memorystore for Low-Latency Systems
25	BigQuery Analytics and Workload Engineering
26	BigLake, Cloud Storage Lakehouse, and Dataplex Governance
27	Pub/Sub and Dataflow Streaming Engineering
28	Managed Service for Apache Spark and Batch Data Processing
29	Database, CDC, and Bulk Data Migration Engineering
30	Data Security, Resilience, AI Readiness, and Enterprise Capstone

Chapter 21 - Cloud SQL and AlloyDB Production Engineering

Choosing and operating managed relational databases with evidence-based resilience

Learning Objectives

- Select Cloud SQL or AlloyDB from compatibility, performance, scale, availability, and operational requirements.
- Design private connectivity, identity, connection pooling, backups, replicas, and disaster recovery.
- Operate schema changes, maintenance, observability, and incident response.
- Validate failover and restore instead of treating configuration as proof.

21.1 Service Selection and Responsibility

Cloud SQL is a managed relational service for MySQL, PostgreSQL, and SQL Server. AlloyDB for PostgreSQL targets demanding PostgreSQL-compatible workloads with a disaggregated architecture, read pools, automatic storage management, and advanced performance capabilities. Neither service removes application-level responsibilities such as schema quality, transaction design, connection behavior, query efficiency, and recovery testing.

Decision Area	Cloud SQL	AlloyDB for PostgreSQL
Compatibility	Managed MySQL, PostgreSQL, or SQL Server	PostgreSQL-compatible workloads requiring higher scale or performance
Availability	Regional HA option; replicas can have HA	Regional primary uses active and standby nodes; read pools scale reads
Read Scaling	Read replicas managed by the application	Read pool instances distribute read capacity
Migration Fit	Broad lift-and-shift target	Modernized PostgreSQL destination and analytical acceleration
Operations	Engine-specific flags, maintenance, backup, PITR	Cluster/instance model, managed pooling, performance tools

21.2 Connectivity, Identity, and Connection Management

Use private IP or Private Service Connect where supported and appropriate. Public IP requires deliberate authorized-network or connector controls and should not become the default. Database IAM authentication can reduce password dependence for supported engines, but database-native privileges still govern what a connected identity can do.

- Use a dedicated runtime service account and the Cloud SQL Auth Proxy or language connector where it improves identity and certificate handling.
- Keep secrets in Secret Manager when database-native passwords remain necessary.
- Use connection pools with bounded concurrency; serverless scale-out can otherwise exhaust database connections.
- Separate administrative, migration, application, read-only, and monitoring identities.

21.3 Availability, Replication, Backup, and DR

Regional HA protects against selected instance or zonal failures. Read replicas offload reads but are not automatically equivalent to DR failover targets. Cross-region replicas or AlloyDB secondary clusters support

regional recovery patterns, while backups and point-in-time recovery protect against deletion, corruption, and operator error. Recovery objectives determine which controls are required.

Control	Primary Purpose	Validation
Regional HA	Reduce interruption from zonal/instance failure	Run a controlled failover and measure reconnect time
Read replica/read pool	Read scale and workload isolation	Verify lag, routing, and consistency expectations
Automated backup/PITR	Recover from logical error	Restore into an isolated target and reconcile data
Cross-region replica/secondary	Regional disaster recovery	Practice promotion, DNS/config switch, and failback
Deletion protection	Prevent accidental removal	Test change permissions and break-glass workflow

21.4 Performance and Change Operations

Monitor CPU, memory pressure, storage, connections, replication delay, transaction latency, lock waits, and slow queries. Query tuning and schema design normally produce better outcomes than blind vertical scaling. Maintenance windows, flags, machine changes, and major upgrades require release plans and compatibility tests.

```
gcloud sql instances describe app-db --
format='yaml(state,region,databaseVersion,settings.availabilityType,settings.backupConfiguration,settings.ipConfiguration)'

gcloud sql operations list --instance=app-db --limit=10
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Connections spike and requests fail	Unbounded pools, serverless concurrency, or leaked connections	Inspect connection metrics and application pools; cap concurrency, reuse connectors, and add backpressure.
Replica lag grows	Write burst, long transaction, insufficient capacity, or network issue	Measure lag and workload; reduce long transactions, scale appropriately, and route latency-sensitive reads carefully.
Failover completes but application stays down	Stale connections, DNS/cache, driver retry, or session assumptions	Inspect client errors and pool behavior; implement bounded retry with jitter and recreate connections.
Restore is incomplete	Wrong backup/PITR timestamp, missing objects, or untested runbook	Restore separately, compare row counts/checksums and critical objects, then update the runbook.

Hands-on Lab 21: Build and Validate a Private HA Relational Service

Prerequisites

- A VPC with private connectivity, a synthetic dataset, gcloud, and permissions for database, network, IAM, and Secret Manager operations.

Implementation

1. Create a small HA Cloud SQL PostgreSQL instance or a suitable AlloyDB learning cluster with deletion protection and automated backup/PITR.
2. Configure private connectivity and a dedicated runtime service account; avoid embedding a database password in source code.
3. Create an application schema and load synthetic orders and customers.
4. Run representative reads/writes through a bounded connection pool and capture latency, connections, and query evidence.
5. Create read-scaling capacity where budget permits and validate routing and replication delay.
6. Trigger a supported controlled failover or maintenance simulation and record application recovery behavior.
7. Restore a backup or PITR copy into an isolated target and reconcile critical data.

Validation Evidence

- No public exposure is required for the application path.
- HA, backup, and deletion-protection settings are documented.
- The application reconnects after failover within the measured objective.
- The restored copy passes row-count and sample checksum validation.

Cleanup

- Delete replicas, restored instances, databases, secrets, private connections, and temporary projects after evidence is saved.

Interview Questions and Model Answers

1. When should you choose AlloyDB instead of Cloud SQL?

When PostgreSQL compatibility is acceptable and the workload needs AlloyDB-specific performance, read scaling, or architecture benefits that justify migration and cost.

2. Is a read replica a backup?

No. Replication can copy logical errors; backups and PITR preserve recovery points for different failure modes.

3. Why can serverless compute overload a database?

Compute may scale faster than database connection capacity, so pools, concurrency limits, and backpressure are essential.

4. What proves database resilience?

Measured failover, reconnect, restore, reconciliation, and documented recovery evidence - not configuration alone.

Chapter Summary

- Cloud SQL and AlloyDB serve different compatibility and performance profiles.
- Private identity-based connectivity and bounded connection pools are production requirements.

- HA, replication, backup, and DR address different failures.
- Recovery must be exercised and reconciled.

Chapter 22 - Spanner Global Relational Engineering

Designing transactional systems that scale across regions without abandoning relational integrity

Learning Objectives

- Explain Spanner instances, configurations, processing units/nodes, databases, and SQL dialects.
- Design keys and schemas that avoid hotspots and support locality.
- Choose read, transaction, and multi-region patterns from consistency and latency requirements.
- Operate capacity, backups, change streams, observability, and recovery.

22.1 Architecture and Configuration Choice

Spanner is a fully managed relational database that combines transactional consistency with horizontal scale. Instance configuration determines regional, dual-region, or multi-region placement and therefore affects write latency, availability, data residency, and cost. The database supports GoogleSQL and PostgreSQL dialect choices, but dialect selection is a long-lived application decision.

Configuration	Strength	Trade-off
Regional	Lowest write latency near one region; zonal resilience	Regional outage requires separate DR strategy
Dual-region	Data placement across two regions with quorum design	Higher write latency and configuration constraints
Multi-region	Broad availability and geographically distributed reads	Highest complexity, latency, and cost
Geo-partitioning	Place selected data nearer jurisdictions/users	Requires placement schema and multi-region foundation

22.2 Schema, Primary Keys, and Hotspot Prevention

Spanner divides data by key ranges. A monotonically increasing leading key can funnel writes into a narrow range and create a hotspot. Use high-cardinality distributed keys, UUIDs with appropriate strategy, hash prefixes, or carefully designed composite keys. Interleaving/locality features can improve related-row access but should follow measured query patterns.

- Make the primary key serve distribution and access-path goals, not only uniqueness.
- Use secondary indexes deliberately; each index adds storage and write work.
- Avoid unbounded transactions and large mutations.
- Review query plans and Key Visualizer before scaling compute blindly.

22.3 Transactions, Reads, and Application Semantics

Read-write transactions provide serializable behavior with external consistency. Read-only transactions and stale reads can reduce latency or improve throughput when exact freshness is not required. Applications should use official client libraries and session pools rather than creating sessions per request.

Operation	Use	Design Note
Read-write transaction	Atomic business changes	Keep short; retry aborts using client guidance

Operation	Use	Design Note
Strong read	Latest committed data	May cost more latency across distance
Bounded/exact stale read	Analytics, feeds, reference data	State acceptable staleness in the product contract
Partitioned DML	Large bulk update/delete	Different semantics; not a general transaction replacement
Change stream	Downstream CDC and integration	Design retention, consumers, and replay

22.4 Capacity, Recovery, and Operations

Size compute for normal demand and failure headroom. Autoscaling can adjust capacity, but schema hotspots, lock contention, inefficient queries, or regional latency cannot be repaired by capacity alone. Backups, point-in-time recovery where configured, exports, and regional architecture should be matched to recovery objectives.

```
gcloud spanner instances describe global-orders
gcloud spanner databases ddl describe orders --instance=global-orders
gcloud spanner backups list --instance=global-orders
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
High CPU on one split or node	Hot key range or skewed access	Use Key Visualizer, query statistics, and key distribution; redesign the leading key or traffic pattern.
Transaction abort rate rises	Contention or long read-write transactions	Inspect lock/transaction metrics; shorten transactions, reduce shared-row updates, and use client retries.
Multi-region writes are slower than expected	Leader distance and quorum latency	Measure from client regions; revisit configuration and leader-aware placement assumptions.
Queries consume excess capacity	Missing index, full scan, poor predicate, or stale statistics	Inspect query plan/statistics; add justified indexes and rewrite access paths.

Hands-on Lab 22: Model and Load a Hotspot-Resistant Spanner Database

Prerequisites

- A learning project, Spanner API, a small regional instance or emulator where appropriate, and synthetic order events.

Implementation

- Create a Spanner instance and database using a selected SQL dialect.
- Define customer, order, and order-item tables with a distributed primary-key strategy.
- Load synthetic data with a deliberately bad sequential-key test and capture its distribution characteristics.
- Load the redesigned schema and compare latency and distribution evidence.
- Run a short read-write transaction, a strong read, and a stale analytical read.

6. Create a backup and document restore steps and recovery ownership.
7. Review query plans and produce a capacity and failure-headroom note.

Validation Evidence

- The schema explanation connects keys to distribution and query patterns.
- Transaction retries and idempotency are demonstrated.
- A backup exists and restoration prerequisites are documented.
- The final design avoids the observed sequential hotspot.

Cleanup

- Delete the database, backup, and instance after exporting non-sensitive evidence.

Interview Questions and Model Answers

1. Why are sequential primary keys risky in Spanner?

They can concentrate new writes in one key range, creating a hotspot despite overall horizontal capacity.

2. What is external consistency?

Transactions appear ordered consistently with real time, enabling globally consistent transactional behavior.

3. When are stale reads useful?

When the product accepts bounded freshness in exchange for lower latency, higher throughput, or locality.

4. Why keep failure headroom?

A surviving configuration must absorb traffic when a zone or region is unavailable; normal-state utilization alone is insufficient.

Chapter Summary

- Configuration determines locality, availability, latency, and cost.
- Primary-key design is a scaling control.
- Transactions must be short, retryable, and semantically idempotent.
- Capacity, schema, and recovery evidence are operated together.

Chapter 23 - Firestore Document Database Engineering

Building serverless application data models without losing query, consistency, or security discipline

Learning Objectives

- Choose Firestore edition, mode, location, and data model from application requirements.
- Design documents, collections, indexes, transactions, and security boundaries.
- Prevent hotspots, uncontrolled fan-out, and expensive query patterns.
- Operate backups, TTL, observability, and migration safely.

23.1 Editions, Modes, and Workload Fit

Firestore is a serverless document database with strong consistency and transactions. Current offerings include Standard and Enterprise editions, with capabilities and query interfaces that differ. Select the edition and access mode only after validating client compatibility, indexing behavior, transaction requirements, and migration constraints.

Good Fit	Question Before Choosing
Mobile/web application state	Can the access patterns be expressed as indexed document queries?
User profiles, catalogs, workflow documents	Are document size and update contention controlled?
Real-time listeners	Can clients tolerate reconnect, fan-out, and billing behavior?
Flexible JSON/BSON-style models	Is schema evolution governed instead of becoming accidental?

23.2 Document Modeling and Index Engineering

Model for the queries that the application must execute. Deep nesting, large arrays, ever-growing documents, and high-frequency updates to a single document can create operational or cost problems. Denormalization is normal, but duplication requires explicit update and reconciliation logic.

- Use stable collection/document paths and avoid encoding mutable business meaning into IDs.
- Exclude fields from indexing when they are never queried and create composite indexes only from real query evidence.
- Use subcollections for unbounded child sets rather than growing one document forever.
- Distribute high-rate writes; avoid sequential IDs and single-document counters without sharding.

23.3 Transactions, Concurrency, and Idempotency

Transactions retry when concurrent changes invalidate their read set. Keep transactions small and ensure the application code can run more than once. Batched writes are useful when reads are not needed. Distributed counters, queues, and workflows require idempotency keys and reconciliation rather than assuming a client sends each request once.

23.4 Security Rules, IAM, Backup, and Operations

IAM controls administrative and server-side access. Security Rules protect supported client access and must be tested as code. Rules are authorization logic, not data validation by themselves. Use App Check where relevant, audit administrative changes, configure TTL only for intended expiration, and restore backups into isolated environments before declaring recovery ready.

```
gcloud firestore databases list
gcloud firestore indexes composite list --database='(default)'
gcloud firestore backups list --location=LOCATION
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Query requires an index	Composite index is missing or edition query rules differ	Review the query error/link, create only the required index, and validate build completion.
High latency or throttling on writes	Hot document, sequential key, or high index fan-out	Inspect document paths and index count; shard counters, distribute IDs, and reduce unused indexes.
Client receives permission denied	Security Rules, auth claims, App Check, or project mismatch	Use the rules emulator/test suite and inspect identity claims and database ID.
Unexpected cost increase	Listener fan-out, repeated reads, broad queries, or index growth	Use usage metrics and query tracing; narrow listeners, paginate, cache, and remove unused indexes.

Hands-on Lab 23: Build a Secure Firestore Order Workflow

Prerequisites

- A Firebase/Google Cloud test project, Firestore database, emulator tooling where possible, and synthetic users/orders.

Implementation

1. Create a database in a deliberate location and record edition/mode decisions.
2. Model users, orders, and order events with bounded documents and query-driven indexes.
3. Write Security Rules that allow a user to read only owned orders while a trusted backend service account performs fulfillment updates.
4. Create emulator tests for allowed and denied operations.
5. Implement an idempotent order submission with a transaction or batched write.
6. Generate concurrent updates to demonstrate retries or contention behavior.
7. Create a backup or export plan, TTL policy for temporary events, and a cost-risk note.

Validation Evidence

- Rule tests include positive and negative cases.
- The order operation remains correct when retried.
- Queries use intentional indexes and pagination.
- Backup/retention and deletion ownership are documented.

Cleanup

- Delete test collections, indexes, backups/exports, and the project when no longer required.

Interview Questions and Model Answers

1. How should Firestore schemas be designed?

From required queries and update patterns, with controlled denormalization and explicit reconciliation.

2. What is the difference between IAM and Security Rules?

IAM governs administrative/server identities; Security Rules authorize supported client data access based on request and document context.

3. Why must transaction functions be idempotent?

Firestore can retry them after contention, so side effects must not be duplicated.

4. What commonly causes Firestore write hotspots?

Sequential identifiers, high-rate updates to one document, and excessive index fan-out.

Chapter Summary

- Firestore is serverless, but data-model and cost decisions remain architectural.
- Indexes are part of the write path and must be governed.
- Security Rules require automated allow/deny tests.
- Transactions, listeners, backup, and TTL need operational evidence.

Chapter 24 - Bigtable and Memorystore for Low-Latency Systems

Engineering wide-column persistence and in-memory acceleration for predictable latency

Learning Objectives

- Select Bigtable for high-throughput key-range workloads and Memorystore for cache/state acceleration.
- Design Bigtable row keys, column families, replication, and application profiles.
- Design cache-aside, TTL, eviction, persistence, and failure behavior.
- Observe hotspots, cache effectiveness, capacity, and durability trade-offs.

24.1 Bigtable Workload and Schema Model

Bigtable is a wide-column database designed for low-latency access at very large scale. It is not a relational database and does not provide joins or arbitrary secondary-index behavior. Rows are sorted lexicographically by row key, so the key must distribute traffic while preserving the scans the application needs.

Pattern	Row-Key Direction	Risk
Time series by device	hash/device + reversed or bucketed time	A single hot device or newest-time hotspot
User timeline	user ID + time bucket + event ID	Large user partitions and unbounded scans
IoT telemetry	region/hash + device + timestamp	Poor prefix choice prevents efficient scan
Lookup table	stable distributed business key	Over-modeling relational access

24.2 Clusters, Replication, App Profiles, and Autoscaling

A Bigtable instance contains clusters; replication copies data between clusters and can increase availability or isolate workloads. Application profiles control routing and consistency behavior. Each cluster needs enough capacity for application traffic plus replication work, including during failover. Autoscaling is usually preferred, but minimum and maximum nodes must reflect performance and recovery objectives.

- Use multi-cluster routing only when its consistency behavior matches the workload.
- Use single-cluster routing for strict locality or workload isolation where required.
- Monitor hottest-node CPU, latency, storage utilization, replication latency, and failed request rates.
- Backups protect recovery points; replication alone does not protect against logical deletion.

24.3 Memorystore Choices and Cache Architecture

Memorystore provides managed Valkey, Redis Cluster, Redis, and a deprecated Memcached offering. New design should evaluate Valkey and Redis variants from client compatibility, sharding, persistence, high availability, cross-region needs, and product lifecycle. A cache must be treated as disposable unless persistence and recovery requirements explicitly say otherwise.

Pattern	Benefit	Control
Cache-aside	Simple and resilient to cache loss	TTL, stampede protection, source-of-truth fallback
Write-through	Cache stays warm	Failure ordering and dual-write correctness

Pattern	Benefit	Control
Session/state store	Low latency shared state	HA, persistence, token expiry, fail-closed behavior
Rate limit/counter	Atomic fast operations	Clock/window semantics and regional consistency
Vector/context cache	Fast retrieval for AI apps	Tenant isolation, TTL, sensitive-data policy

24.4 Hotspot, Stampede, and Durability Operations

Bigtable hotspots and cache stampedes are different manifestations of skew. Use key distribution, request coalescing, jittered TTLs, negative caching, bounded concurrency, and circuit breakers. Memorystore replicas and persistence complement each other: replicas address node/zonal failure; persistence addresses wider data-loss scenarios according to the selected product.

```
gcloud bigtable instances list
gcloud bigtable clusters list --instances=telemetry
gcloud redis clusters list --region=REGION
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Bigtable latency rises on hottest node	Skewed row key or uneven workload	Inspect Key Visualizer/hottest-node metrics; redesign prefixes or split traffic.
Replication lag or failover overload	Clusters undersized for replication and surviving traffic	Scale each cluster for failure state and test app-profile routing.
Database load spikes when cache expires	Cache stampede or synchronized TTL	Add jitter, single-flight/request coalescing, stale-while-revalidate, and rate limits.
Cache memory remains high	Missing TTL, oversized values, fragmentation, or policy mismatch	Inspect memory/eviction metrics and key sampling; set TTLs, size limits, and correct eviction strategy.

Hands-on Lab 24: Build a Time-Series Store with a Resilient Cache

Prerequisites

- Bigtable and a small Memorystore learning instance where available, synthetic telemetry, and a private VPC.

Implementation

- Design two candidate Bigtable row keys and explain scan and hotspot behavior.
- Create a table with bounded column families and garbage-collection policies.
- Load synthetic telemetry and run point and range reads through an application profile.
- Enable autoscaling or document the capacity envelope; capture hottest-node and latency evidence.
- Implement cache-aside reads in Memorystore with jittered TTL and negative caching.
- Simulate cache loss and confirm the application falls back safely without overwhelming Bigtable.

7. Create a Bigtable backup and document cache persistence/HA decisions separately.

Validation Evidence

- The selected row key supports the required scan and distributes load.
- GC policies and backup protect retention/recovery objectives.
- Cache hit ratio and fallback behavior are measured.
- Cache failure does not corrupt authoritative data.

Cleanup

- Delete Bigtable tables, backups, clusters/instances, Memorystore resources, and test networking.

Interview Questions and Model Answers

1. Why is Bigtable row-key design critical?

Rows are ordered by key, so the key controls data locality, scan efficiency, and whether traffic hotspots.

2. Does Bigtable replication replace backups?

No. Replication improves availability but can replicate logical mistakes; backups provide recovery points.

3. What is a cache stampede?

Many clients simultaneously miss or expire the same key and overload the source of truth.

4. Why evaluate Valkey for new Memorystore designs?

Google Cloud now offers a managed Valkey service, so lifecycle, compatibility, features, and operational requirements should be compared rather than assuming one cache product.

Chapter Summary

- Bigtable scales key-range workloads when row keys distribute traffic.
- Replication, app profiles, capacity, and backup are separate controls.
- Caches require explicit miss, expiry, eviction, and failure behavior.
- Measure hotspots and hit ratio before adding capacity.

Chapter 25 - BigQuery Analytics and Workload Engineering

Building governed analytical systems with predictable performance and cost

Learning Objectives

- Explain BigQuery storage, jobs, locations, slots, reservations, editions, and on-demand pricing.
- Design tables with partitioning, clustering, and lifecycle controls.
- Engineer query performance, workload isolation, governance, and reliability.
- Use BigQuery as an AI-ready analytical foundation without exposing uncontrolled data.

25.1 Serverless Analytics Architecture

BigQuery separates durable analytical storage from distributed query compute. Users submit jobs to load, export, query, copy, transform, or manage data. Dataset location is an architectural boundary because jobs and connected services must respect location compatibility. Cost can be based on bytes processed or capacity reservations, depending on the operating model.

Pricing/Capacity Model	Best Fit	Control
On-demand analysis	Variable or exploratory workloads	Maximum bytes billed, partition filters, dry runs, budgets
Reservations and editions	Steady or isolated enterprise workloads	Assignments, baseline/autoscale slots, commitments
BI Engine/accelerators	Interactive dashboards	Capacity sizing and cache effectiveness
External/BigLake tables	Open data in object storage	Metadata, file layout, governance, and scan cost

25.2 Table Design and Data Lifecycle

Partition by a column that appears in real filters, usually event or ingestion time. Cluster on columns that improve block pruning for repeated predicates. Avoid creating thousands of tiny tables as a substitute for partitioning. Define expiration, retention, and legal-hold behavior at dataset and table levels.

- Require partition filters for large fact tables where practical.
- Use appropriate data types and nested/repeated fields rather than serializing everything as strings.
- Keep raw, curated, and serving layers with explicit ownership and contracts.
- Use snapshots/clones/copies deliberately for testing and recovery, not as unmanaged duplication.

25.3 Query Performance and Workload Isolation

Query plans reveal stages, shuffle, skew, and slot consumption. Reduce scanned data, filter early, aggregate before large joins, avoid repeated transformations, and materialize stable intermediate results when justified. Reservations can isolate production, development, ETL, and data-science workloads while allowing controlled idle-slot sharing.

```
bq query --use_legacy_sql=false --dry_run \
  'SELECT customer_id, SUM(amount) FROM `PROJECT.sales.orders` WHERE order_date >= DATE_SUB(CURRENT_DATE(),
  INTERVAL 30 DAY) GROUP BY customer_id'

bq show --format=prettyjson PROJECT:sales.orders
```

25.4 Governance, Sharing, ML, and AI Readiness

Use dataset/table IAM, authorized views and routines, row-level security, column-level policy tags, data masking, audit logs, and service perimeters according to classification. BigQuery ML and vector/search capabilities can bring model work near governed data, but training and retrieval datasets still require consent, quality, lineage, evaluation, and leakage controls.

AI-Ready Control	Evidence
Data contract and quality	Freshness, completeness, uniqueness, validity, drift metrics
Lineage and provenance	Source, transformations, model/retrieval usage, owner
Sensitive-data protection	Policy tags, masking, row restrictions, approved service perimeter
Cost isolation	Reservation/assignment, labels, quotas, bytes limits
Reproducibility	Versioned SQL, snapshots, model parameters, evaluation set

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Query scans too much data	Missing partition filter, broad SELECT, or external-file layout	Use dry run and query plan; add filters, select columns, partition/cluster, compact files.
Dashboard latency is unstable	Competing workloads, repeated heavy query, or poor model	Inspect slot and job timelines; isolate reservations, aggregate, cache, or materialize.
Access denied to table/column	Dataset IAM, policy tag, row policy, or perimeter restriction	Inspect effective IAM, Data Catalog/Universal Catalog policy tags, row policies, and VPC SC logs.
Costs rise unexpectedly	Exploratory scans, unbounded scheduled queries, streaming, or idle capacity	Use billing export, INFORMATION_SCHEMA, labels, reservations, query quotas, and ownership.

Hands-on Lab 25: Build a Governed BigQuery Analytics Mart

Prerequisites

- A BigQuery project, synthetic sales files, and permissions for datasets, jobs, IAM, and policy tags where available.

Implementation

- Create raw and curated datasets in a deliberate location with default expiration and labels.
- Load synthetic orders into a partitioned and clustered table.
- Run an unoptimized query and capture bytes, plan stages, and runtime; then optimize it.
- Create a scheduled or scripted transformation that is idempotent and partition-aware.
- Create a restricted analyst view and apply row/column controls to a sensitive field.

6. Use a dry run and maximum-bytes guardrail; document on-demand versus reservation choice.
7. Produce a small BigQuery ML or vector-ready feature table with quality checks and lineage notes.

Validation Evidence

- Partition pruning and reduced bytes are visible.
- The analyst cannot access protected raw/sensitive data.
- Transformation reruns do not duplicate data.
- The AI-ready dataset has an owner, quality evidence, and approved fields.

Cleanup

- Delete datasets, reservations/assignments, scheduled queries, policy tags, and temporary files after evidence is exported.

Interview Questions and Model Answers

1. What is a BigQuery slot?

A unit of compute capacity used by query execution; reservations pool and allocate slots to workloads.

2. When should a table be partitioned?

When a common selective filter can divide data into manageable partitions and improve lifecycle and scan control.

3. Why can clustering help?

It organizes storage blocks by selected columns, enabling pruning for repeated predicates within partitions or tables.

4. What makes analytics data AI-ready?

Governed provenance, quality, classification, reproducibility, approved usage, and cost/security controls - not merely availability.

Chapter Summary

- BigQuery architecture separates storage and analytical compute.
- Partitioning, clustering, and query design control both performance and cost.
- Reservations isolate workloads and allocate capacity.
- AI and ML features must remain inside data-governance boundaries.

Chapter 26 - BigLake, Cloud Storage Lakehouse, and Dataplex Governance

Operating open data across object storage and analytics engines with unified governance

Learning Objectives

- Design raw, curated, and serving zones in Cloud Storage.
- Use BigLake and open table formats without losing access control or metadata discipline.
- Apply Dataplex Universal Catalog, lineage, quality, and discovery capabilities.
- Create an AI-ready lakehouse with lifecycle, small-file, and governance controls.

26.1 Lakehouse Architecture and Object Layout

A lakehouse stores open or semi-open analytical data in object storage while exposing governed tables to query and processing engines. Cloud Storage bucket location, object naming, file format, compression, partition layout, retention, and lifecycle policies are foundational choices. Too many tiny files increase metadata and processing overhead; very large files reduce parallelism and recovery flexibility.

Layer	Purpose	Controls
Landing/raw	Immutable source capture	Write-once convention, checksum, source metadata, retention
Standardized	Validated schema and formats	Quarantine, deduplication, partitioning, quality results
Curated	Business entities and metrics	Data contracts, ownership, SCD/reconciliation
Serving/feature	BI, APIs, ML/RAG inputs	Access policy, freshness SLO, evaluation and rollback

26.2 BigLake and Open Table Formats

BigLake extends governed analytics over data in object storage and supports lakehouse patterns, including Apache Iceberg-oriented workflows. Separate the physical data from the catalog and policy layer. Writers must coordinate schema and partition evolution, snapshot retention, compaction, and concurrent updates.

- Use columnar formats such as Parquet for analytical scans when appropriate.
- Keep catalog ownership and table mutation rights narrow.
- Schedule compaction and expiration of obsolete snapshots/files according to recovery requirements.
- Test interoperability with every engine that will read or write the table.

26.3 Dataplex Universal Catalog, Lineage, and Quality

Dataplex Universal Catalog provides discovery and governance capabilities across data assets. Catalog entries, aspects/metadata, policy tags, lineage, profiling, and data-quality results turn storage into an operated platform. Metadata must be part of the deployment pipeline rather than an optional documentation exercise.

Governance Object	Operational Question
Owner/steward	Who approves access and fixes quality?

Governance Object	Operational Question
Classification/policy tag	What data is sensitive and how is it protected?
Lineage	Which sources and transformations produced this field?
Quality rule	What threshold blocks promotion or alerts the owner?
Freshness SLO	How late can the dataset be before consumers are notified?
Retention/legal hold	When may objects and snapshots be deleted?

26.4 AI-Ready Data Products

Retrieval and model pipelines amplify data defects and access mistakes. Curate approved corpora and features, preserve document/source identifiers, remove or mask prohibited content, track embedding/model versions, and create evaluation datasets. A vector index is not a governance boundary; the underlying content and retrieved output remain subject to classification and deletion requirements.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
External table is slow	Small files, unselective scan, poor partitioning, or remote metadata overhead	Inspect file counts/size and query plan; compact, partition, cache/materialize, or load into native storage.
Table metadata and files disagree	Concurrent writers or failed commit/cleanup	Inspect catalog snapshots and object history; stop writers, reconcile metadata, and restore a known snapshot.
Users cannot discover trusted data	Missing catalog metadata, ownership, or certification	Publish entries, aspects, owner, quality, lineage, and usage guidance.
Sensitive data appears in AI retrieval	Classification/masking or corpus filtering failed	Disable the serving path, inspect lineage and index contents, remove data, re-embed, and add policy tests.

Hands-on Lab 26: Build a Governed Lakehouse Data Product

Prerequisites

- Cloud Storage, BigQuery/BigLake access, Dataplex permissions, and synthetic customer event files.

Implementation

- Create landing, standardized, curated, and serving prefixes or buckets with location, retention, versioning, and lifecycle decisions.
- Write synthetic events as partitioned Parquet and demonstrate the impact of many small files.
- Create a governed external/BigLake table and query it from BigQuery.
- Register metadata in Dataplex Universal Catalog with owner, classification, description, and freshness target.
- Create data-quality checks for nulls, uniqueness, valid ranges, and referential expectations.
- Create or capture lineage from landing through curated output.
- Prepare an approved AI/RAG corpus subset with prohibited fields removed and a deletion/rebuild runbook.

Validation Evidence

- File layout and compaction evidence are recorded.
- Catalog entries expose owner, classification, lineage, and quality.
- Unauthorized users cannot bypass governed table access to sensitive data.
- The AI corpus can be rebuilt after a source deletion.

Cleanup

- Delete tables, catalog entries, scans, data, buckets, and temporary processing resources.

Interview Questions and Model Answers

1. What problem does BigLake solve?

It provides governed analytical access to data in object storage and supports lakehouse interoperability patterns.

2. Why are small files harmful?

They increase listing, metadata, scheduling, and per-file overhead across query and Spark engines.

3. What makes lineage operationally useful?

It identifies affected downstream assets during incidents, schema changes, access reviews, and deletion requests.

4. Why is a vector index not a security boundary?

It can reproduce information derived from protected content; source policy, access, deletion, and evaluation still apply.

Chapter Summary

- Object layout and lifecycle are architecture decisions.
- Open table formats require catalog and writer coordination.
- Dataplex metadata, quality, and lineage make data discoverable and governable.
- AI-ready data products require rebuild, deletion, provenance, and evaluation controls.

Chapter 27 - Pub/Sub and Dataflow Streaming Engineering

Designing event pipelines with explicit delivery, time, state, and replay semantics

Learning Objectives

- Design Pub/Sub topics, subscriptions, schemas, retention, dead letters, and cross-project ownership.
- Build idempotent subscribers for at-least-once delivery and use exactly-once features only when justified.
- Explain Beam windows, watermarks, triggers, state, and Dataflow streaming modes.
- Operate backlog, lag, autoscaling, draining, replacement, replay, and cost.

27.1 Pub/Sub Resource and Delivery Model

Publishers write messages to topics; each subscription represents an independent delivery stream. Pull, push, export, and BigQuery-oriented patterns differ in control and failure handling. At-least-once delivery means subscribers must tolerate duplicates. Exactly-once delivery is available for supported pull subscriptions within documented boundaries, but it adds coordination and should not replace idempotent business logic.

Control	Purpose	Common Mistake
Schema	Validate message shape/encoding	Treating schema compatibility as business validation
Retention/seek/snapshot	Replay and recovery	No owner or capacity plan for replay storm
Dead-letter topic	Isolate repeatedly failing messages	Never reprocessing or alerting on DLQ growth
Ordering key	Order related messages	Using one key for all traffic and serializing throughput
Flow control	Protect subscriber capacity	Unlimited in-flight messages and memory pressure

27.2 Idempotency, Ordering, and Failure Contracts

Use a stable event ID and a durable idempotency record or naturally idempotent upsert. Acknowledgment should follow successful durable processing. Retries must use backoff, and poison messages must eventually move to a dead-letter path with diagnostics. Ordering is per ordering key and can reduce availability or increase latency during recovery.

- Separate event time, publish time, and processing time.
- Include schema version, producer, correlation ID, tenant, and classification metadata.
- Do not put secrets or unrestricted personal data in message attributes.
- Define replay effects before enabling long retention.

27.3 Apache Beam and Dataflow Semantics

Dataflow executes Apache Beam pipelines for batch and streaming. Windows group unbounded events, watermarks estimate event-time progress, and triggers decide when results are emitted. Allowed lateness and accumulation mode determine how late data changes prior outputs. Stateful processing requires careful keying and bounded state.

Concept	Question
Window	Which events belong to one result?
Watermark	How complete does the system believe event time is?
Trigger	When should early/on-time/late output be emitted?
Allowed lateness	How long may late events update a result?
State/timer	What per-key memory and timeout logic is required?
Streaming mode	Exactly-once default or at-least-once cost/latency trade-off?

27.4 Operations, Update, and Replay

Monitor oldest unacknowledged message age, backlog bytes/messages, delivery attempts, dead-letter rate, Dataflow system lag, watermark, worker utilization, throughput, and sink errors. Drain preserves in-flight processing for supported streaming shutdown; cancel stops more abruptly. Pipeline replacement must preserve state compatibility or use a planned migration.

```
gcloud pubsub subscriptions describe orders-sub
gcloud pubsub subscriptions seek orders-sub --time='2026-07-30T00:00:00Z'
gcloud dataflow jobs list --region=REGION
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Backlog grows continuously	Subscriber capacity, downstream bottleneck, errors, or flow control	Inspect oldest age, ack rate, worker/sink metrics; scale or fix downstream before replay.
Duplicate business effects	Subscriber not idempotent or ack occurs before durable commit	Use event IDs/upserts, transactionally record processing, and ack after success.
Late events change totals unexpectedly	Window, watermark, trigger, or allowed-lateness mismatch	Inspect event timestamps and Beam configuration; align semantics with product contract.
Dataflow will not scale	Key skew, fused bottleneck, quota, worker config, or sink limit	Inspect stage and autoscaling metrics; reshuffle keys, separate stages, raise quota, or protect sink.

Hands-on Lab 27: Build an Idempotent Streaming Orders Pipeline

Prerequisites

- Pub/Sub, Dataflow, BigQuery, a service account, and synthetic order-event generator.

Implementation

- Create a versioned Pub/Sub schema, topic, primary subscription, and dead-letter topic/subscription.
- Publish synthetic orders including duplicates, late events, and one poison message.
- Build a Beam pipeline with event-time windows, idempotent writes, and a quarantine path.
- Run it on Dataflow with a bounded worker range and appropriate streaming mode.

5. Observe backlog, watermark, system lag, throughput, and dead-letter behavior.
6. Drain or replace the job using a documented release procedure.
7. Seek/replay a controlled time range into an isolated output and reconcile counts.

Validation Evidence

- Duplicates do not create duplicate business records.
- Late-event behavior matches the documented window contract.
- Poison messages enter a monitored dead-letter path.
- Replay is isolated, capacity-controlled, and reconciled.

Cleanup

- Stop/drain jobs; delete topics, subscriptions, schemas, tables, buckets, service accounts, and temporary images/templates.

Interview Questions and Model Answers

1. Why must Pub/Sub subscribers be idempotent?

At-least-once delivery can redeliver messages, and network/ack failures make duplicates unavoidable.

2. What is a watermark?

An estimate of event-time completeness used by Beam to reason about when windows are likely complete.

3. When choose Dataflow at-least-once streaming mode?

When duplicates are acceptable or deduplicated elsewhere and lower cost/latency is worth the trade-off.

4. What is the difference between drain and cancel?

Drain processes in-flight data and finalizes supported state; cancel stops the job more abruptly.

Chapter Summary

- Messaging guarantees must be translated into business idempotency.
- Event time and processing time are different.
- Beam windows, watermarks, and triggers define result correctness.
- Replay and pipeline replacement are production change events.

Chapter 28 - Managed Service for Apache Spark and Batch Data Processing

Operating Spark as a governed data product rather than a collection of unmanaged clusters

Learning Objectives

- Choose serverless or cluster-based managed Spark execution.
- Design storage, dependencies, metadata, service accounts, and network access.
- Tune partitioning, shuffle, autoscaling, and file output.
- Orchestrate, observe, troubleshoot, and retire batch workloads safely.

28.1 Managed Spark Deployment Models

Google Cloud Managed Service for Apache Spark includes serverless and cluster deployment options. Serverless batches and interactive sessions remove cluster lifecycle work and charge during execution. Cluster deployments provide deeper control for long-running, specialized, or ecosystem-heavy workloads. Select from workload duration, startup tolerance, dependency needs, isolation, networking, and cost.

Model	Best Fit	Operational Concern
Serverless batch	Periodic ETL, notebooks-to-jobs, elastic workloads	Startup, quotas, dependency packaging, per-job identity
Interactive session	Exploration with managed execution	Idle time, data access, reproducibility
Ephemeral cluster	Complex job with custom cluster config	Provisioning, image/version, teardown
Long-lived cluster	Shared services or low-startup-latency workload	Patching, contention, idle cost, tenant isolation

28.2 Storage, Metastore, and Dependency Design

Keep durable data in Cloud Storage, BigQuery, Bigtable, or governed lakehouse tables rather than relying on cluster disks. Use Dataproc Metastore or compatible catalogs where shared Hive metadata is required. Pin runtime and library versions, store artifacts in controlled repositories, scan dependencies, and separate job identity from human identity.

28.3 Performance Engineering

Spark performance is dominated by input layout, partition count, skew, shuffle, serialization, executor sizing, and output-file strategy. More workers can make a badly partitioned job more expensive without making it faster.

- Use column pruning and predicate pushdown.
- Repartition or salt skewed keys before wide joins.
- Broadcast only genuinely small dimensions.
- Avoid collecting large datasets to the driver.
- Compact output to useful file sizes and avoid one-file-per-task explosions.
- Capture Spark UI/event logs for post-run diagnosis.

28.4 Orchestration, Security, and Reliability

Use Workflows or Cloud Composer where multi-step dependencies, retries, sensors, and scheduling require orchestration. Jobs should be restartable from checkpoints or immutable inputs, write to temporary locations, validate output, then publish atomically. Private networking, narrow service accounts, CMEK where required, and controlled egress apply to Spark just as to application compute.

```
gcloud dataproc batches submit pyspark gs://BUCKET/jobs/curate.py \
--region=REGION \
--service-account=spark-job@PROJECT.iam.gserviceaccount.com \
--subnet=projects/HOST/regions/REGION/subnetworks/data-subnet
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Job is slow with low CPU	Input bottleneck, tiny files, insufficient parallelism, or blocked I/O	Inspect Spark stages, input files, task duration, and connector metrics; compact or repartition.
Few tasks are extremely slow	Data skew or straggler executor	Inspect key distribution and stage task metrics; salt/repartition and handle hot keys.
Driver runs out of memory	collect/toPandas, large broadcast, or excessive metadata	Remove driver collection, bound results, adjust broadcast, and compact file listings.
Output contains duplicates after retry	Job is not restartable/idempotent	Write to run-specific staging, validate, then atomically publish or MERGE with run ID.

Hands-on Lab 28: Run and Tune a Restartable Spark Curation Job

Prerequisites

- A Cloud Storage bucket, Managed Service for Apache Spark access, a private subnet if required, and synthetic event data.

Implementation

1. Generate partitioned input plus a deliberately skewed key and many small files.
2. Package a PySpark job with pinned dependencies and a dedicated service account.
3. Run a serverless batch and capture stage/task metrics and event logs.
4. Fix skew and small-file behavior, rerun, and compare runtime and cost evidence.
5. Write output to a run-specific staging path, validate row counts/checksums, then publish.
6. Schedule or orchestrate the job with retries that do not duplicate output.
7. Document when a cluster deployment would be justified instead.

Validation Evidence

- The optimized job improves measured stage behavior.
- The service account has only required storage/catalog/logging permissions.
- A failed/retried run cannot corrupt the published dataset.
- Output file sizes and partitions are appropriate for consumers.

Cleanup

- Delete batches, staging/output data, workflow/schedule, service accounts, and any clusters or metadata services created for the lab.

Interview Questions and Model Answers

1. When choose serverless Spark?

For elastic batch or interactive work where avoiding cluster lifecycle outweighs startup or customization constraints.

2. What usually causes Spark skew?

A small number of keys or partitions hold a disproportionate amount of data and work.

3. Why store event logs?

They enable post-run analysis of stages, tasks, shuffle, skew, and failures after compute is gone.

4. How make a Spark job restartable?

Use immutable/checkpointed inputs, run-specific staging, deterministic transformations, validation, and atomic publication or idempotent merge.

Chapter Summary

- Managed Spark deployment should match workload and operational needs.
- Data layout and shuffle dominate performance.
- Job identity, dependencies, and network access require governance.
- Restartability and atomic publication are more important than automatic retries alone.

Chapter 29 - Database, CDC, and Bulk Data Migration Engineering

Moving operational and analytical data with measurable fidelity and controlled cutover

Learning Objectives

- Plan migrations with discovery, compatibility, security, performance, and rollback evidence.
- Use Database Migration Service for supported one-time and continuous database migrations.
- Use Datastream for CDC into BigQuery, Cloud Storage, and custom pipelines.
- Use Storage Transfer Service for governed bulk object migration and replication.

29.1 Migration Factory and Wave Planning

A migration is a product change, not a copy command. Inventory sources, versions, extensions, size, growth, peak change rate, dependencies, owners, downtime tolerance, RTO/RPO, residency, and licensing. Group systems into repeatable waves and prove the pattern on a representative low-risk workload before critical cutover.

Phase	Exit Evidence
Discover	Inventory, dependency map, classification, owners, objectives
Assess	Compatibility report, target sizing, network and cost model
Prepare	Landing zone, identities, connectivity, schema conversion, runbooks
Replicate/transfer	Initial load complete, lag and error thresholds met
Validate	Counts, checksums, samples, performance, permissions, business sign-off
Cut over	Write freeze/redirect, final sync, application switch, monitoring
Stabilize/retire	Rollback window closed, source archived/decommissioned, savings verified

29.2 Database Migration Service

Database Migration Service streamlines supported migrations into Cloud SQL and AlloyDB. Continuous migration performs an initial load followed by ongoing change replication, enabling minimal downtime when source and destination are synchronized. Heterogeneous migrations add schema and code conversion risk, so stored procedures, data types, collations, extensions, and application SQL require explicit testing.

29.3 Datastream CDC and Downstream Processing

Datastream is a serverless change data capture and replication service. It can deliver operational changes to BigQuery, Cloud Storage, Apache Iceberg-oriented destinations, or custom Dataflow pipelines depending on source/destination support. Datastream delivery is at least once, so downstream merge and deduplication logic must use source keys, transaction metadata, and ordering expectations.

- Measure data freshness from source commit to target availability.
- Include backfill and CDC reconciliation; do not assume the handoff is gap-free.
- Handle source schema change and unsupported object types.
- Protect logs and change records because CDC can expose deleted or sensitive values.

29.4 Storage Transfer and Cutover Control

Storage Transfer Service moves or synchronizes large data sets from supported cloud and on-premises sources to Cloud Storage and between buckets. Define overwrite/delete behavior, checksums, include/exclude filters, bandwidth windows, service-agent permissions, and object metadata preservation. Physical appliances or network transfer may be appropriate for constrained networks.

```
gcloud database-migration migration-jobs list --region=REGION
gcloud datastream streams list --location=REGION
gcloud transfer jobs list --project=PROJECT_ID
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Migration lag never reaches threshold	Source change rate exceeds replication, network bottleneck, or large transaction	Inspect CDC/replication metrics; increase capacity, reduce load, tune network, or change cutover plan.
Row counts match but application fails	Schema objects, collations, sequences, permissions, or SQL behavior differ	Run compatibility tests and business transactions; compare objects and execution plans, not only rows.
CDC target has duplicates	At-least-once delivery and non-idempotent merge	Use source primary key and change metadata to deduplicate/upsert.
Bulk transfer repeatedly retransfers objects	Timestamp/checksum/overwrite policy or metadata differences	Inspect transfer logs and options; define authoritative side and comparison behavior.

Hands-on Lab 29: Execute a Minimal-Downtime Migration Rehearsal

Prerequisites

- A small source database, a Cloud SQL/AlloyDB destination, Datastream or DMS support as appropriate, and synthetic data.

Implementation

- Create an inventory and migration decision record including downtime, rollback, data classification, and acceptance thresholds.
- Establish private or encrypted connectivity and dedicated migration identities.
- Run compatibility/schema assessment and remediate unsupported objects.
- Start initial load and continuous replication; generate controlled source changes.
- Measure lag, validate counts/checksums/critical queries, and run application smoke tests.
- Perform a rehearsal cutover with write control, final synchronization, application switch, and enhanced monitoring.
- Trigger rollback within the agreed window, then repeat the forward cutover and document source retirement.

Validation Evidence

- Data and schema fidelity checks pass.
- Lag and cutover duration are measured against objectives.
- Application smoke tests pass on the target.

- Rollback and forward cutover are both executable from the runbook.

Cleanup

- Delete migration jobs, streams, connection profiles, temporary databases, transfer jobs, buckets, and credentials.

Interview Questions and Model Answers

1. What is continuous migration?

An initial snapshot/load followed by ongoing replication until final cutover, reducing downtime.

2. Why are row counts insufficient validation?

They do not prove schema objects, ordering, constraints, permissions, encoding, sequences, or business behavior.

3. What guarantee should Datastream consumers assume?

At-least-once delivery, requiring idempotent downstream processing.

4. When is rollback no longer safe?

After the new target accepts writes that cannot be reliably replayed back to the source or after the agreed rollback window closes.

Chapter Summary

- Migration success is measured by application and data fidelity.
- Continuous replication reduces downtime but increases coordination.
- CDC consumers must deduplicate and reconcile.
- Every cutover needs explicit rollback limits and source-retirement controls.

Chapter 30 - Data Security, Resilience, AI Readiness, and Enterprise Capstone

Integrating governance, recovery, observability, and delivery into one production data platform

Learning Objectives

- Apply defense in depth to identities, networks, encryption, perimeters, and data access.
- Define data SLOs, quality, lineage, retention, backup, and regional recovery.
- Create production incident and change-management practices for data systems.
- Deliver a portfolio-grade AI-ready enterprise data platform capstone.

30.1 Data Security Architecture

Use separate projects for platform control, ingestion, processing, analytics, and sensitive workloads where risk justifies it. Apply IAM groups and service accounts, short-lived federation, Secret Manager, private connectivity, VPC Service Controls, organization policy, audit logs, and customer-managed keys only with a reliable key lifecycle. Protect data in every copy: source, CDC, raw lake, staging, backup, cache, feature table, vector index, export, and logs.

Layer	Required Evidence
Identity	Group/service-account ownership, least privilege, no unmanaged keys
Network	Private paths, controlled egress, DNS, service perimeter design
Data access	Dataset/table/row/column policies and tested denial
Encryption	Default or CMEK decision, key owner, rotation and recovery
Audit	Admin and data-access logs appropriate to classification
Exfiltration	Export controls, perimeters, anomaly monitoring, incident runbook

30.2 Reliability, Quality, and Recovery Objectives

Data-platform SLOs include freshness, completeness, accuracy, availability, latency, and successful publication. Recovery design must address infrastructure failure, regional outage, corruption, accidental deletion, bad schema, poison events, failed migration, and security compromise. Replicas improve availability; backups and immutable copies provide recovery points; replay reconstructs derived data only when sources and code are retained.

Failure	Primary Control	Proof
Zonal/instance failure	HA/replication	Failover test and client recovery
Regional failure	Cross-region design/DR target	Promotion and traffic-switch game day
Logical deletion/corruption	Backup/PITR/snapshot	Isolated restore and reconciliation
Bad pipeline release	Versioned code/data and rollback	Canary, replay, and output comparison
Source removal request	Lineage and rebuild/delete workflow	Affected assets removed and revalidated

Failure	Primary Control	Proof
Credential compromise	Short-lived identity, audit, perimeter	Revoke, rotate, investigate, and restore trust

30.3 Operability and Incident Command

Create dashboards and alerts for database saturation, replication lag, failed backups, Pub/Sub backlog, Dataflow lag, Spark failures, BigQuery cost/slot pressure, quality violations, catalog freshness, and access anomalies. Incidents need a commander, operations lead, communications lead, timeline, hypotheses, evidence, containment, recovery, and follow-up actions. Avoid destructive changes before preserving logs and state.

30.4 Enterprise Capstone Architecture

The capstone builds a multi-project retail data platform. Operational orders use a managed relational database. Product/catalog documents use Firestore where appropriate. High-volume telemetry uses Bigtable. Pub/Sub and Dataflow deliver real-time events to BigQuery. Cloud Storage and BigLake hold governed open data, Dataplex publishes metadata and quality, and managed Spark curates large batches. Datastream/DMS supports migration. An approved feature and retrieval corpus supplies AI workloads through controlled datasets.

- Foundation: resource hierarchy, budgets, IAM groups, service accounts, APIs, audit, private networks, service perimeters.
- Data plane: operational stores, streaming path, lakehouse, warehouse, cache, catalog, and backup locations.
- Delivery: infrastructure and data definitions as code, schema compatibility checks, quality gates, canary/replay strategy.
- Operations: SLO dashboards, on-call runbooks, restore evidence, regional game day, cost allocation, retention and deletion.
- Portfolio: architecture diagram, threat model, ADRs, runbooks, test evidence, cost model, and executive summary.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Data pipeline is green but business data is wrong	Quality rules do not cover semantic defect or source contract changed	Stop publication, quarantine output, inspect lineage/source change, restore prior version, and add a contract test.
Regional recovery succeeds but consumers fail	DNS, identities, keys, quotas, dependencies, or catalogs not recovered	Use dependency map and game-day checklist; pre-provision and test all control-plane dependencies.
CMEK-protected data becomes unavailable	Key disabled/deleted, IAM lost, or region mismatch	Engage key owner, preserve evidence, restore key access/version according to break-glass process.
AI output reveals restricted information	Corpus/index policy, retrieval filtering, or access boundary failed	Disable retrieval, inspect lineage and logs, delete/rebuild affected index, notify security/privacy owners, and test controls.

Hands-on Lab 30: Final Enterprise Google Cloud Data Platform Capstone

Prerequisites

- Completion of Chapters 21-29 labs or equivalent skills, a dedicated capstone project structure, synthetic data, and a strict budget.

Implementation

1. Write requirements, data classification, SLOs, RTO/RPO, residency, cost ceiling, and architecture decision records.
2. Build the minimum viable multi-project foundation with private networks, service accounts, audit, budgets, and policy controls.
3. Deploy one relational store, one high-scale NoSQL/time-series pattern, a cache, Pub/Sub/Dataflow stream, BigQuery mart, and governed lakehouse zone.
4. Implement metadata, lineage, quality gates, retention, backup, and deletion workflows.
5. Migrate a synthetic legacy source using DMS or Datastream and perform a rehearsed cutover/rollback.
6. Create an approved AI-ready feature or retrieval dataset with sensitive-field removal, provenance, evaluation, and rebuild instructions.
7. Run failure exercises: duplicate event, poison message, bad schema, database failover, restore, and one regional dependency simulation.
8. Produce an operations handbook, evidence pack, cost report, and executive architecture review.

Validation Evidence

- All data paths have an owner, classification, identity, network path, retention, and recovery control.
- Quality and schema gates prevent a deliberately bad release.
- Restore and replay evidence meets the declared objectives.
- The platform can identify and remove a prohibited source record from derived AI assets.
- Resources stay within the budget and can be fully destroyed from the documented inventory.

Cleanup

- Export sanitized portfolio evidence, then delete all compute, database, stream, transfer, analytics, catalog, key, network, and storage resources. Confirm billing and asset inventory are empty.

Interview Questions and Model Answers

1. What is the most important distinction in data resilience?

Availability, replication, backup, replay, and disaster recovery solve different failure modes and require separate tests.

2. How do you prevent a data platform from becoming an access-control bypass?

Apply consistent identity, network, classification, row/column policy, perimeter, export, audit, and deletion controls to every copy and derivative.

3. What should a senior engineer present in an architecture review?

Requirements, trade-offs, failure modes, security, cost, operability, migration, evidence, and explicit residual risks.

4. What makes this capstone portfolio-grade?

It demonstrates production decisions and measured evidence across architecture, delivery, security, reliability, cost, migration, and operations - not only screenshots of created services.

Chapter Summary

- Security applies to every data copy and derivative.
- Data SLOs include correctness and freshness, not only uptime.
- Recovery and deletion must cover analytics and AI-derived assets.
- The capstone integrates architecture, delivery, migration, operations, security, and cost evidence.

Part C Integrated Practice Program

Complete the following practice sequence after the chapter labs. Keep an evidence repository containing architecture decisions, commands, sanitized screenshots, test outputs, incident timelines, cost reports, and cleanup confirmation.

Week	Focus	Evidence
1	Cloud SQL/AlloyDB and Spanner selection, private access, failover, restore	Decision matrix, connection test, failover/restore timing
2	Firestore, Bigtable, Memorystore modeling and hotspot tests	Schema/key ADRs, rule tests, cache failure evidence
3	BigQuery and lakehouse optimization/governance	Query plans, bytes/cost delta, catalog/quality/lineage
4	Pub/Sub/Dataflow and managed Spark	Duplicate/late-event tests, Spark tuning comparison
5	DMS/Datastream/Storage Transfer rehearsal	Compatibility, lag, fidelity, cutover and rollback
6	Capstone security, game day, AI readiness, portfolio review	Threat model, SLOs, recovery evidence, executive summary

Additional Enterprise Projects

Project	Required Outcome
Global financial ledger prototype	Spanner with distributed keys, strong transactions, change stream, backup, and regional architecture review.
Real-time fraud feature platform	Pub/Sub, Dataflow, Bigtable/BigQuery, online cache, quality and replay controls.
Governed healthcare lakehouse simulation	Synthetic data, BigLake, Dataplex classification/lineage, row/column protection, deletion workflow.
Retail analytics modernization	Datastream CDC into BigQuery, curated marts, cost isolation, dashboard SLOs, rollback.
IoT telemetry platform	Bigtable time-series design, autoscaling, multi-cluster routing, Spark compaction, retention.
AI retrieval corpus factory	Approved documents, policy filtering, lineage, chunk/version metadata, evaluation and full rebuild.
Multi-cloud object migration	Storage Transfer Service with checksums, metadata, bandwidth window, reconciliation, and source retirement.
Database modernization assessment	Compare Cloud SQL, AlloyDB, Spanner, and Firestore against one application with quantified trade-offs.
Data incident response game day	Simulate leaked credentials, bad pipeline release, corrupted partition, and backup restoration.
FinOps data platform	BigQuery billing export, labels, reservation analysis, lifecycle savings, unit-cost dashboard.

Senior Interview Drill Bank

Scenario	Strong Answer Direction
A Cloud SQL primary is healthy but the application times out after a traffic surge.	Investigate connection exhaustion and pool behavior before resizing; protect the database with bounded concurrency and backpressure.
Spanner CPU is normal overall but p99 latency rises.	Check hottest-node/key-range distribution, lock contention, query plans, client locality, and per-method metrics.
Firestore cost doubled with no user growth.	Inspect listener/query patterns, document reads, index fan-out, retries, and background scans.
Bigtable multi-cluster routing returns unexpected stale data.	Review application-profile consistency/routing choices and replication state; use single-cluster routing where semantics require it.
BigQuery daily spend spikes.	Use billing export, INFORMATION_SCHEMA jobs, labels, bytes processed, scheduled queries, and reservation/assignment changes to identify owner and cause.
A Dataflow job has low CPU and rising backlog.	Find sink bottleneck, key skew, external service limits, flow control, or fused stage blocking rather than adding workers blindly.
A Spark job is fast but creates 200,000 files.	Fix output partitioning and compaction; performance is not acceptable if downstream metadata and query cost explode.
Datastream target has duplicates.	This is compatible with at-least-once delivery; implement key/change-metadata based deduplication or MERGE.
A regional DR test fails because the KMS key is unavailable.	The control plane was not included in the dependency model; recover/replicate key strategy and retest end to end.
An AI index contains a deleted customer record.	Trace lineage, disable serving if required, delete/rebuild derived artifacts, verify absence, and improve deletion propagation tests.

Command Reference

Area	Examples
Cloud SQL	<code>gcloud sql instances list</code> ; <code>gcloud sql operations list --instance=NAME</code>
AlloyDB	<code>gcloud alloydb clusters list --region=REGION</code> ; <code>gcloud alloydb instances list --cluster=CLUSTER --region=REGION</code>
Spanner	<code>gcloud spanner instances list</code> ; <code>gcloud spanner databases list --instance=INSTANCE</code>
Firestore	<code>gcloud firestore databases list</code> ; <code>gcloud firestore indexes composite list</code>
Bigtable	<code>gcloud bigtable instances list</code> ; <code>gcloud bigtable clusters list --instances=INSTANCE</code>

Area	Examples
Memorystore	<code>gcloud redis instances list --region=REGION</code> ; <code>gcloud redis clusters list --region=REGION</code>
BigQuery	<code>bq ls</code> ; <code>bq show PROJECT:DATASET.TABLE</code> ; <code>bq query --dry_run --use_legacy_sql=false SQL</code>
Pub/Sub	<code>gcloud pubsub topics list</code> ; <code>gcloud pubsub subscriptions describe NAME</code>
Dataflow	<code>gcloud dataflow jobs list --region=REGION</code> ; <code>gcloud dataflow jobs describe JOB --region=REGION</code>
Managed Spark	<code>gcloud dataproc batches list --region=REGION</code> ; <code>gcloud dataproc batches describe ID --region=REGION</code>
DMS	<code>gcloud database-migration migration-jobs list --region=REGION</code>
Datastream	<code>gcloud datastream streams list --location=REGION</code>
Storage Transfer	<code>gcloud transfer jobs list --project=PROJECT_ID</code>
Asset and cleanup	<code>gcloud asset search-all-resources --scope=projects/PROJECT_ID</code>

Command groups and flags evolve. Always confirm the active gcloud component and official command reference before production use. Run describe/list commands first, and verify project, region, identity, and deletion impact.

Production Readiness Checklist

Architecture

- ☐ Workload access patterns and consistency are documented.
- ☐ Service choice includes rejected alternatives and cost/failure trade-offs.
- ☐ Location, residency, network, and dependency boundaries are explicit.

Security

- ☐ Human and workload identities are separate and least-privileged.
- ☐ Private access and controlled egress are used where required.
- ☐ Classification, row/column controls, secrets, keys, perimeters, and audit are tested.

Reliability

- ☐ HA, read scaling, backup, PITR, replay, and DR responsibilities are distinct.
- ☐ Failover, restore, reconciliation, and regional dependency tests have evidence.
- ☐ Capacity includes failure-state headroom and quota planning.

Data Engineering

- ☐ Schemas, keys, partitions, indexes, windows, and file layout are validated with realistic load.
- ☐ Pipelines are idempotent, restartable, versioned, and observable.
- ☐ Quality, lineage, contracts, freshness, retention, and deletion are enforced.

Operations

- ☐ Dashboards and alerts connect to owners and runbooks.

- ☐ Changes use canary/rehearsal/rollback where possible.
- ☐ Cost allocation, budgets, labels, lifecycle, cleanup, and unit economics are reviewed.

AI Readiness

- ☐ Approved sources and purposes are documented.
- ☐ Sensitive/prohibited fields are excluded or protected.
- ☐ Feature/corpus provenance, version, evaluation, deletion, and rebuild are tested.

Glossary

Term	Definition
At-least-once delivery	A delivery model in which a message can be delivered more than once; consumers must be idempotent.
Change data capture (CDC)	Capturing inserts, updates, and deletes after an initial state to propagate changes.
Clustering	Organizing BigQuery storage blocks by selected columns to improve pruning.
Data contract	An owned agreement for schema, semantics, quality, freshness, and compatibility.
Data lineage	Traceability from sources through transformations to outputs and consumers.
External consistency	A transaction ordering guarantee consistent with real-time ordering, provided by Spanner.
Hotspot	A disproportionate concentration of traffic on one key range, document, partition, or resource.
Idempotency	The property that repeating an operation does not create additional unintended effects.
Lakehouse	An architecture combining object-storage openness with governed table and analytics capabilities.
PITR	Point-in-time recovery to a selected moment within the configured recovery window.
Policy tag	A classification/control mechanism used to protect sensitive columns and integrate governance.
Reservation	A BigQuery pool of slot capacity assigned to workloads.
Row key	The lexicographically ordered primary identifier that determines Bigtable distribution and scans.
Schema evolution	Controlled change of data structure while preserving compatible producers and consumers.
Slot	A unit of BigQuery computational capacity.
Watermark	A Beam estimate of progress through event time.
Window	A grouping of unbounded events for aggregation.
Workload Identity Federation	Short-lived access from external or workload identities without downloaded service-account keys.

Official Documentation Index

Service	Official Documentation
Cloud SQL	https://docs.cloud.google.com/sql/docs
AlloyDB for PostgreSQL	https://docs.cloud.google.com/alloydb/docs
Spanner	https://docs.cloud.google.com/spanner/docs
Firestore	https://docs.cloud.google.com/firestore/docs
Bigtable	https://docs.cloud.google.com/bigtable/docs
Memorystore	https://docs.cloud.google.com/memorystore/docs
BigQuery	https://docs.cloud.google.com/bigquery/docs
BigLake	https://docs.cloud.google.com/biglake/docs
Dataplex Universal Catalog	https://docs.cloud.google.com/dataplex/docs
Pub/Sub	https://docs.cloud.google.com/pubsub/docs
Dataflow	https://docs.cloud.google.com/dataflow/docs
Managed Service for Apache Spark	https://docs.cloud.google.com/managed-spark/docs
Database Migration Service	https://docs.cloud.google.com/database-migration/docs
Datastream	https://docs.cloud.google.com/datastream/docs
Storage Transfer Service	https://docs.cloud.google.com/storage-transfer/docs
Cloud Architecture Center - Data	https://docs.cloud.google.com/architecture/data-analytics

Cloud services change continuously. Recheck editions, regions, limits, compatibility, deprecations, pricing, and command syntax against official documentation before production implementation.

END OF VOLUME 8C | NEXT: VOLUME 8D - GOOGLE CLOUD CONTAINERS, SERVERLESS, APIS, EVENTS, AND WORKFLOWS